《计算机组成原理》

实验报告



班级：2318039

姓名：魏朝旭

学号：23009201082

日期：2024.12.22

MIPS 汇编实验报告

一、 实验内容

本次实验使用 MIPS 汇编语言完成了以下任务：

① 打印“Hello MIPS”，实现了简单的文本输出功能，展示了 MIPS 汇编对字符串的处理能力。

② 完成两个数的加、减、乘、除法运算并输出运算结果，涵盖了基本算术运算在 MIPS 汇编中的实现，有助于理解数据运算的底层逻辑。

③ 进行多个数的加、减、乘、除法运算并输出运算结果，进一步拓展了数据处理的复杂度，对寄存器管理和运算顺序控制提出了更高要求。

④ 设计加、减、乘、除四则运算计算器，实现了一个具有交互性的计算工具，能够根据用户输入的操作符和数字进行相应运算并输出结果，体现了较为完整的程序逻辑和人机交互设计。

二、需求分析

① 软件的功能、性能要求

能够在合理的时间内完成各项运算任务，避免因算法效率低下导致的长时间等待。合理规划寄存器的使用，避免因寄存器冲突或浪费而影响程序性能。实现数据的准确输入、输出，确保用户输入的数据能够被正确读取和处理，运算结果能够清晰地展示给用户。提供友好的人机交互界面，通过合理的提示信息引导用户进行操作，提升用户体验。

② 作为验收依据，选取测试用例及对应验收数据测试用例。

1.对于打印 “Hello MIPS”，验收标准为控制台准确输出 “Hello MIPS” 字符串。

2.两个数运算的测试用例及验收数据如下：

选取 15 和 35 进行加法作为测试用例，预期结果为 50。

3.多个数运算的测试用例及验收数据如下：

选取 10、20、30 作为加法测试用例，预期结果为 60。

4. 四则运算计算器的测试用例及验收数据如下：

“3+3=6，4-2=2，3/3=1，2*5=10”四组测试用例。

三、代码及相关分析

（一）打印 “Hello MIPS”

```
.text
.globl main
main:
    la $a0,str    # 将字符串“hello world/n”的地址加载到 $a0 寄存器
    li $v0, 4     # 设置系统调用号为 4，表示打印字符串
    syscall       # 执行系统调用，打印出字符串

    li $v0,10     # 设置系统调用号为 10，表示程序正常结束
    syscall
.data
str:.asciiz "Hello MIPS/n"
```

在这段代码中，.data 段定义了一个字符串常量 str。.text 段的 main 函数里，先将字符串地址加载到 $a0 寄存器，再通过设置系统调用号为 4 并执行 syscall 指令来打印字符串。最后设置系统调用号为 10 使程序正常结束。

（二）两个数的运算

```
.text
.globl main
main:
    la $t0,varl
    lw $t1,0($t0)
    lw $t2,4($t0)
    add $t3,$t1,$t2

    sw    $t3,8($t0)
    li $v0,1
        move $a0,$t3
    syscall
    li $v0,10
```

```
        syscall

    .data
    varl:.word 15, 35
```

在.data 段定义了变量 varl 存储两个 32 位整数 15 和 35 。在.text 段的 main 函数中，la 指令获取 varl 的地址到 $t0，lw 指令分别读取两个数到 $t1 和 $t2，add 指令进行加法运算并将结果存到 $t3。然后将结果存储回内存，设置系统调用号为 1 并将结果移到 $a0 寄存器后执行 syscall 打印结果，最后设置系统调用号为 10 结束程序。

（三）多个数的运算

```
.text
.globl main
main:
    la $t0, varl
    lw $t1, 0($t0)
    lw $t2, 4($t0)
    lw $t3, 8($t0)
    add $t4, $t1, $t2
    add $t5, $t4, $t3
    sw $t5, 12($t0)
    li $v0, 1
    move $a0, $t5
    syscall

    li $v0, 10
    syscall

.data
    varl:.word 10, 20, 30
```

.data 段定义了 varl 存储三个整数 10、20 和 30 。main 函数中，先加载地址并读取三个数到 $t1、$t2 和 $t3，通过两次 add 指令完成三个数相加并将结果存到 $t5，存储结果到内存后，设置系统调用号为 1 并将结果移到 $a0 寄存器执行 syscall 打印结果，最后结束程序。

（四）四则运算计算器

```
.data
```

```
    value: .word 0, 0, 0, 0, 0        # 0: first num, 4: second num, 8: operation, 12: result
    msg0            :    .asciiz    " please choose the operation(1~4):\n\t\t1 : +,addition \n\t\t2 : -,subtracter\n\t\t3 : * multiplication \n\t\t4 : /,division\n"
    msg1            :    .asciiz    "first num:"
    msg2            :    .asciiz    "second num:"
    addFlag         :    .asciiz    " + "
    subFlag         :    .asciiz    " - "
    mulFlag         :    .asciiz    " * "
    divFlag         :    .asciiz    " / "
    equalStr        :    .asciiz    " = "
    newline         :    .asciiz    "\n==================================\n"
.text
.globl main
main:
    la $t0, value               # 加载 value 地址到 $t0
    la $a0,msg0
    li $v0,4
    syscall                     # 打印操作选择提示

    li   $v0, 5
    syscall
    sw   $v0, 8($t0)            # 输入操作符

    la $a0,msg1
    li $v0,4
    syscall                     # 打印第一个数输入提示

    li   $v0, 5
    syscall
    sw   $v0, 0($t0)            # 读取第一个数

    la $a0,msg2
    li $v0,4
    syscall                     # 打印第二个数输入提示

    li   $v0, 5
    syscall
    sw   $v0, 4($t0)            # 读取第二个数

    la $a0,newline
    li $v0,4
    syscall                     # 打印换行

    lw $t1, 0($t0)              # 加载第一个数
    lw $t2, 4($t0)              # 加载第二个数
    lw $t3, 8($t0)              # 加载操作符
```

```
    beq $t3,1,addOp          # 根据操作符跳转
    beq $t3,2,subOp
    beq $t3,3,mulOp
    beq $t3,4,divOp

addOp:
    add $t4, $t1, $t2        # 加法运算
    sw $t4, 12($t0)
    la $t5,addFlag
    j printResult
subOp:
    sub $t4, $t1, $t2        # 减法运算
    sw $t4, 12($t0)
    la $t5,subFlag
    j printResult
mulOp:
    mul $t4, $t1, $t2        # 乘法运算
    sw $t4, 12($t0)
    la $t5,mulFlag
    j printResult
divOp:
    div $t4, $t1, $t2        # 除法运算
    sw $t4, 12($t0)
    la $t5,divFlag
    j printResult

printResult:
    lw $a0,0($t0)
    li $v0,1
    syscall                  # 打印第一个数

    la $a0,0($t5)
    li $v0,4
    syscall                  # 打印操作符

    lw $a0,4($t0)
    li $v0,1
    syscall                  # 打印第二个数

    la $a0,equalStr
    li $v0,4
    syscall                  # 打印等号

    lw $a0,12($t0)
    li $v0,1
```

```
    syscall             # 打印运算结果
    j exit

exit:
    la $a0,newline
    li $v0,4
    syscall             # 打印换行

    #j main
    li $v0,10
    syscall             # 退出程序
```

在.data 段定义了一系列用于存储数据和提示信息的变量。.text 段的 main 函数中，首先提示用户选择操作符并读取，然后分别读取两个操作数。根据操作符的值跳转至相应的运算代码块进行运算，并将结果存储到指定位置。最后在 printResult 标签处打印出完整的运算表达式和结果，在 exit 标签处打印换行后结束程序。

四、调试分析

在打印 “Hello MIPS” 时，若出现乱码或未打印出字符串，可能是字符串的定义格式错误，例如字符串结尾缺少正确的换行符或字符串的 asciiz 定义被误写为其他类型。解决方法是仔细检查字符串定义，确保格式正确，并且检查 la 指令加载的地址是否准确指向字符串的起始位置。

对于两个数运算，如果结果错误，可能是数据读取错误，如 lw 指令的偏移量设置错误，导致读取了错误的数据。或者是运算指令使用错误，如将加法指令误写为其他指令。通过在关键指令前后设置断点，查看寄存器的值，对比预期值来排查错误。

多个数运算调试时，若结果不正确，可能是加法顺序错误或者中间结果存储和读取有误。例如，多次加法运算中，寄存器使用混乱，导致数据被覆盖或错误累加。可以通过单步调试，查看每一步寄存器的变化，检查加法逻辑是否正确。

四则运算计算器调试过程中，可能出现运算符判断错误，导致程序跳转至错误的运算代码块。这可能是由于 beq 指令的比较值设置错误或者操作符输入后存储的值与预期不

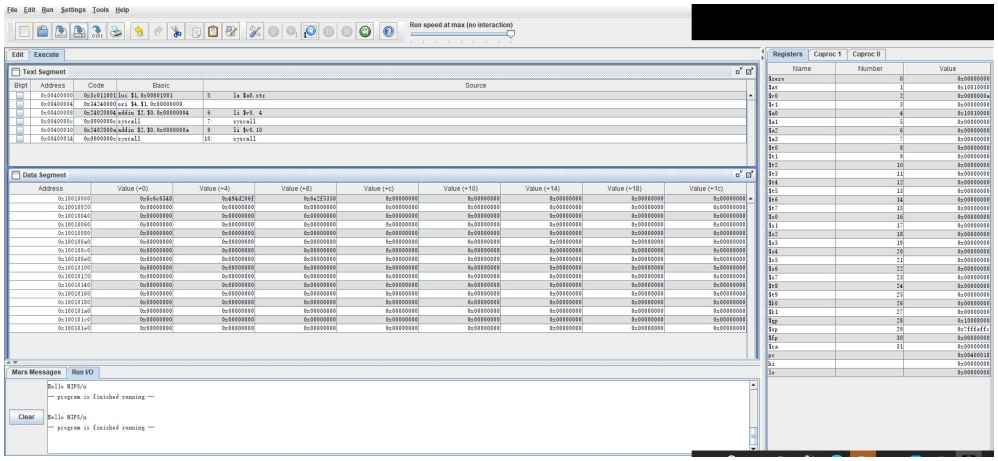
符。需要检查操作符的输入、存储和比较逻辑，确保程序能够正确根据操作符进行跳转和运算。另外，如果除法运算出现除零错误，需要在代码中添加对除数是否为零的判断逻辑，避免程序崩溃。

五、使用说明

本实验的 MIPS 代码需在文本编辑器中编写，编写时要严格遵循 MIPS 汇编语言的语法规则。编写完成后，可以在 Mars 或者 Qtspim 模拟器软件中运行。推荐使用 Mars 模拟器，其具有更友好的界面和强大的调试功能。在运行代码时，根据程序的提示输入相应的数据，如四则运算计算器中需要输入操作符和操作数，然后观察控制台输出的结果是否符合预期。

六、测试结果（以控制台输出和代码窗口截图为准）

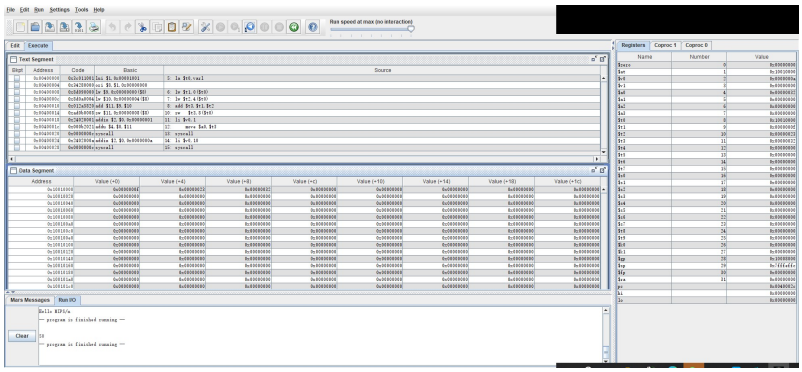
1. 打印“Hello MIPS”：控制台输出“Hello MIPS/n”，符合预期。



2.两个数运算：

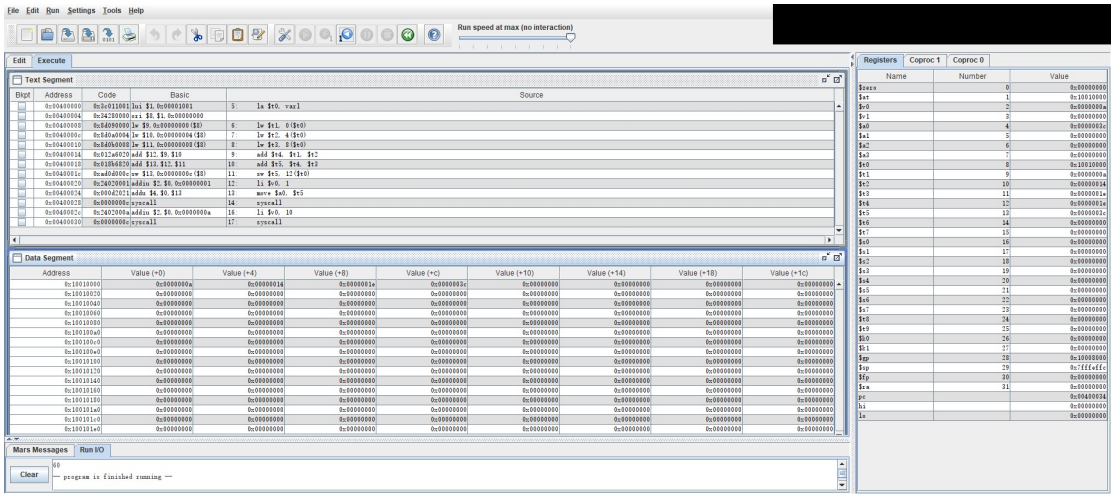
对于加法，输入 15 和 35，输出结果为 50，与预期一致。

减法、乘法、除法运算也分别尝试了测试，结果符合预期。



3.多个数运算：

输入 10、20 和 30 进行加法运算，输出结果为 60，符合预期。



4.四则运算计算器：

“3+3=6，4-2=2，3/3=1，2*5=10”四组测试用例符合预期。

```
please choose the operation(1~4):
                1 : +,addition
                2 : -,subtracter
                3 : * multiplication
                4 : /,division
1
first num:3
second num:3

_____________________

3 + 3 = 6
_____________________

-- program is finished running --
```

```
please choose the operation(1~4):
                1 : +,addition
                2 : -,subtracter
                3 : * multiplication
                4 : /,division
2
first num:4
second num:2

_____________________

4 - 2 = 2
_____________________

-- program is finished running --
```

```
please choose the operation(1~4):
                1 : +,addition
                2 : -,subtracter
                3 : * multiplication
                4 : /,division
4
first num:3
second num:3

_____________________

3 / 3 = 1
_____________________
```

```
please choose the operation(1~4):
                1 : +,addition
                2 : -,subtracter
                3 : * multiplication
                4 : /,division
3
first num:2
second num:5

_____________________

2 * 5 = 10
_____________________
```

通过对各项功能的测试，结果均正确合理，表明本次实验所编写的 MIPS 汇编代码能够满足实验要求，实现了相应的功能。